

REQUEST → RESPONSE

client → web server (NGINX) → WSGI wsgi.py / ASGI asgi.py → HttpRequest → middleware (top→down) → URL resolver (ROOT_URLCONF) → view (+ ORM, templates, forms) → HttpResponse → middleware (bottom→up) → client. Signals: request_started / request_finished. Exceptions: process_exception, handler404/500.

MIDDLEWARE (optional in grey):

SecurityMiddleware | SessionMiddleware | LocaleMiddleware | CommonMiddleware | CsrfViewMiddleware | AuthenticationMiddleware | LoginRequiredMiddleware | MessageMiddleware | XFrameOptionsMiddleware

ContentSecurityPolicyMiddleware

PROJECT & CLI

uv add django # or python -m pip install
django-admin startproject config
./manage.py startapp catalog # INSTALLED_APPS
./manage.py runserver | check | --deploy | test
./manage.py shell # auto-imports models (5.2)
./manage.py createsuperuser | dbshell
./manage.py dumpdata app_model --natural-foreign
./manage.py loaddata books.json | collectstatic
Command: app/management/commands/x.py, class
Command(BaseCommand) (call_command() in tests) +
add_arguments() | handle() | Apps do one thing;
AppConfig.ready() | wires signals.

SETTINGS

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
DEBUG = False; ALLOWED_HOSTS = ["example.com"]
DATABASES = {"default": {"ENGINE":
"django.db.backends.postgresql",
"OPTIONS": {"pool": True}}, # 5.1 pools
STORAGES = {"default": {"BACKEND": "django.core.
files.storage.FileSystemStorage"},
"staticfiles": {"BACKEND": "django.contrib.
staticfiles.storage.
ManifestStaticFilesStorage"}}
from django.conf import settings
Also INSTALLED_APPS | MIDDLEWARE | TEMPLATES
ROOT_URLCONF | STATIC_ROOT | MEDIA_ROOT
AUTH_USER_MODEL | TIME_ZONE | CONN_MAX_AGE;
DEFAULT_AUTO_FIELD = BigAutoField (6.0). Env vars /
split settings. ImproperlyConfigured,
DJANGO_SETTINGS_MODULE: new key
get_random_secret_key() | rotate with
SECRET_KEY_FALLBACKS.

MODELS & FIELDS

class Book(models.Model):
class Status(models.TextChoices):
DRAFT = "m", "Draft"; PUB = "p", "Published"
title = models.CharField(max_length=200)
slug = models.SlugField(unique_for_date="pub")
status = models.CharField(max_length=1,
choices=Status, default=Status.DRAFT)
price = models.DecimalField(max_digits=6,
decimal_places=2, null=True, blank=True)
stock = models.IntegerField(db_default=0) # 5.0
pub = models.DateField()
created = models.DateTimeField(default=Now())
total = models.GeneratedField # 5.0
expression=F("price") * F("stock"),
output_field=models.DecimalField(...),
db_persist=True)
class Meta:
ordering = ["-created"]
constraints = [models.UniqueConstraint(
fields=["title", "author"], name="uq_t_a"),
models.CheckConstraint(
condition=~(price__gte=0), name="price_ok")]
indexes = [models.Index(fields=["slug"])]
def __str__(self): return self.title
def get_absolute_url(self):
return reverse("catalog:book", args=[self.pk])
null = DB, blank = forms (strings: blank, not null).
auto_now, add, JSONField UUIDField FileField
ImageField, db.comment. Meta: verbose_name
db.table unique_together get_latest_by. Composite
PK (5.2): pk = models.CompositePrimaryKey("order",
"line").

RELATIONSHIPS & ON_DELETE

author = models.ForeignKey(Author,
on_delete=models.PROTECT, related_name="books")
user = models.OneToOneField(
settings.AUTH_USER_MODEL, on_delete=CASCADE)
tags = models.ManyToManyField(Tag,
through="tagging") # extra fields
shelf = models.ForeignKey(Shelf,
on_delete=models.DB_CASCADE) # 6.1
Python-side: CASCADE PROTECT RESTRICT SET NULL
SET_DEFAULT SET() DO NOTHING DB-level (6.1):
DB_CASCADE DB_SET_NULL DB_SET_DEFAULT (no signals).
Reverse: author.books.all(). ContentType +
GenericForeignKey: sparingly.

INHERITANCE, MANAGERS, MIGRATIONS

Abstract (Meta.abstract, e.g. TimeStampedModel).
multi-table (implicit 1:1 join; avoid) - proxy (proxy =
True) _meta.get_fields().
class BookQS(models.QuerySet):
def published(self):
return self.filter(status="P")
objects = BookQS.as_manager()
./manage.py makemigrations [--check]
./manage.py migrate [app [0003] zero]
./manage.py squashmigrate app [0004]
./manage.py makemigrations --empty app # data
migrations.RunPython(fwd, reverse_code=back)
in fwd: Book = apps.get_model("catalog", "Book")
Commit migrations, keep them reversible;
SeparateDatabaseAndState; atomic = False for big ops.

ORM QUERIES (LAZY QUERYSETS)

Book.objects.filter(title__icontains="dj",
author_name__startswith="A")
.exclude(status="D")
.order_by("-pub")[1:10]
get() first() exists() count() values() only()
get_or_create() update_or_create() bulk_create()
qs.update(stock=F("stock") - 1) # no race
filter(Q(a=1) | Q(b=2)) # OR / NOT
aggregate(avg=Avg("price")) # -> dict
annotate(n=Count("reviews")).filter(n__gte=5)
new = Review.objects.filter(book__outerRef("pk"))
annotate(last=Subquery(new.values("at")[1:1]))
annotate(r=Window(Rank(), order_by="price"))
Lookups exact iexact contains in lte lt range
isnull startswith year_atlas() Exists Case/When
Coalesce Async: get() acreate(), async for.
raw(sql, {params}): always params.

CHANGED SINCE OLD TUTORIALS (BOOKS TARGET DJANGO 1.11–2.1)

4.0 url() removed → path()/re_path()
5.1 index_together removed → Meta.indexes
3.2 DEFAULT_AUTO_FIELD: 6.0 BigAutoField default

OPTIMISATION & FETCH MODES (6.1)

qs.select_related("author") # FK/1:1 JOIN
qs.prefetch_related(Prefetch("reviews",
queryset=Review.objects.filter(ok=True)))
qs.fetch_mode(models.FETCH_PEEKS)
FETCH_ONE (default) | FETCH_PEEKS | FETCH_RAISE
FETCH_PEEKS loads a lazy field for every instance of the
batch (no N+1); FETCH_RAISE raises FieldFetchBlocked.
iterator() | explain() | measure with
assertNumQueries, Debug Toolbar.

TRANSACTIONS

with transaction.atomic(): # or @atomic
order.save()
transaction.on_commit(lambda:
send_receipt.enqueue(order_id=order.pk))
Nested = savepoint; ATOMIC_REQUESTS per DB;
select_for_update(); using("replica") +
DATABASE_ROUTERS.

URLS

path("books/", include("catalog.urls")) # root
app_name = "catalog" # namespace
urlpatterns =
path("", views.book_list, name="list"),
path("<int:pk>/", views.BookDetail.as_view(),
name="book"),
re_path(r"^/?P<page>[0-9]{4}\$/{}/\$", views.y)
reverse("catalog:book", args=[1]); reverse_lazy()
url "catalog:book" book.pk pk
Converters str int slug uuid path.
register_converter(); handler404/500;
i18n_patterns().

FUNCTION VIEWS & SHORTCUTS

@require_POST # require_http_methods([...])
@login_required # permission_required("a.x")
def publish(request, pk):
book = get_object_or_404(Book, pk=pk)
book.publish()
messages.success(request, "Published")
return redirect("catalog:book", pk=pk)
render(request, "catalog/list.html", ctx)
JsonResponse({"ok": 1}); JsonResponse(status=204)
raise Http404 / PermissionDenied
Also @cache_page, @login_not_required (5.1).
@csrf_exempt (avoid). async def views run natively on
ASGI.

CLASS-BASED & GENERIC VIEWS

Base	View(dispatch, TemplateView, RedirectView)
Display	ListView(paginate_by), DetailView
Edit	FormView CreateView UpdateView DeleteView
Dates	ArchiveIndexView YearArchiveView MonthArchiveView DayArchiveView DateDetailView
Auth	LoginView LogoutView PasswordChangeView PasswordResetView

class Book(LoginRequiredMixin, UpdateView):
model = Book; fields = ["title", "price"]
success_url = reverse_lazy("catalog:list")
def get_queryset(self): # scope to user
return super().get_queryset().filter(
owner=self.request.user)
Hooks: get_queryset set_object get_context_data
get_form kwargs form_valid form_invalid
get_success_url dispatch. Mixins left of the base view;
keep it shallow.

REQUESTS, SESSIONS, FILES, RESPONSES

request.GET POST FILES COOKIES headers user
session; await request.auser().
request.session["cart"] = {...}; messages.info();
signing.dumps(), set_signed_cookie().
FileField(upload_to=...) → STORAGES["default"]
StreamingHttpResponse (async iterators on ASGI 4.2).
FileResponse(cf, as_attachment=True), CSV with
csv.writer(response), PDF with WeasyPrint.

TEMPLATES (DTL)

{% extends "base.html" %} {% load static %}
{% block body %} {{ b.title|upper }}{% endblock %}
{% for b in page_obj %}{% forloop.counter %}
{% empty %}None{% endfor %}
{% if user.is_authenticated %}{% endif %}
{% include "card.html" with book=b only %}
{% url "catalog:book" b.pk %} {% static "a.css" %}
{% csrf_token %} {% with t=b, price %} {% cache %}
{% querystring page=3 %} 5.1
{% partialdef row inline %}{% endpartialdef %}
{% partial row %} renderf, "list.html"row() 6.0
Filters default date length truncatetimes
Linebreaks join safe floatformat pluralize
json_script. Auto-escaping on; format_html over
mark_safe. Context processors request auth messages
csp. No queries in templates. Partials + django-htmx for
fragments; Jinja2 optional.

CUSTOM TAGS & FILTERS, STATIC

app/templates/tags/catalog_tags.py
register = template.Library()
@register.filter
def money(v): return f"€{v:2f} EUR"
@register.simple_tag(takes_context=True)
@register.inclusion_tag("catalog/card.html")
@register.simple_block_tag # 5.2
STATICFILES_DIRS → collectstatic → STATIC_ROOT;
WhiteNoise or a CDN.

FORMS

class BookForm(forms.ModelForm):
class Meta:
model = Book; fields = ["title", "price"]
def clean_title(self): # one field
return self.cleaned_data["title"].strip()
def clean(self): # cross-field
data = super().clean()
self.add_error("price", "Too high")
form = BookForm(request.POST or None, instance=b)
if form.is_valid():
b = form.save(commit=False)
b.owner = request.user; b.save()
form.save_m2m()
Order: to_python() → validate() →
run_validators() → clean<field>() →
Form.clean() → ModelForm.full_clean(); raise
ValidationError: NeverForm.all().exclude. Formsets:
formset_factory, modelform_factory,
inlineformset_factory(Author, Book, fields=[...])
+ management.form.Rendering({ form }) (div),
as_div, { form.title.as_field_group } (6.0),
FORM_RENDERER, widgets + Media.

ADMIN

@admin.register(Book)
class BookAdmin(admin.ModelAdmin):
list_display = ["title", "status", "price_eur"]
list_filter = ["status"]
search_fields = ["title"]
prepopulated_fields = {"slug": ["title"]}
raw_id_fields = ["author"]
date_hierarchy = "pub"
inlines = [ReviewInline] # TabularInline
actions = ["publish"]
@admin.display(description="Price",
ordering="price")
def price_eur(self, obj):
return f"({obj.price}) EUR"
@admin.action(description="Publish selected")
def publish(self, request, queryset):
queryset.update(status="P")
Also autocomplete_fields readonly_fields
fieldsets list_select_related, get_queryset(),
has_change_permission(), facets (5.0), template
overrides, @staff_member_required. Admin is for
staff.

AUTHENTICATION

path("accounts/",
include("django.contrib.auth.urls"))
<form method="post" action="{% url 'logout' %}">
<input type="checkbox" checked="" value="1" /> Log out /</form>
user = authenticate(request, username=u,
password=p)
login(request, user); logout(request)
await authenticate(...); await login(...) # 5.2
Logout is POST-only (5.0). LoginRequiredMiddleware
(5.1) + @login_not_required; LoginRequiredMixin
PermissionRequiredMixin UserPassesTestMixin;
LOGIN_URL LOGIN_REDIRECT_URL
LOGOUT_REDIRECT_URL.

CUSTOM USER & PERMISSIONS

class User(AbstractBaseUser): pass # day one
AUTH_USER_MODEL = "accounts.User"
get_user_model() # in code; FKs use the setting
user.has_perm("catalog.change_book")
Group Permission Meta.permissions. Email login:
AbstractBaseUser + BaseUserManager,
USERNAME_FIELD = "email", custom UserAdmin;
AUTHENTICATION_BACKENDS. Object perms: django-
guardian; social: django-allauth.

PASSWORDS, SECURITY & CSP

PASSWORD, HASHERS = ["django.contrib.auth.hashers",
Argon2PasswordHasher, ...PBKDF2..., ...Crypt...]
SECURE_SSL_REDIRECT = True
SECURE_HSTS_SECONDS = 31536000
SESSION_COOKIE_SECURE = CSRF_COOKIE_SECURE = True
CSRF_TRUSTED_ORIGINS = ["https://example.com"]
SECURE_PROXY_SSL_HEADER = (
"HTTP_X_FORWARDED_PROTO", "https") # trusted
from django.utils.csp import CSP # 6.0
SECURE_CSP = {"default-src": [CSP.SELF],
"script-src": [CSP.SELF, CSP.NONCE]}
<script nonce="{% csp_nonce %}">
First hasher hashes, the rest verify;
AUTH_PASSWORD_VALIDATORS check password CSP:
ContentSecurityPolicyMiddleware,
SECURE_CSP.REPORT_ONLY, context_processors.csp.
check --deploy: DEBUG, weak SECRET_KEY,
ALLOWED_HOSTS, SSL redirect, HSTS, secure cookies,
X_FRAME_OPTIONS. JS sends X-CSRFToken; ORM params,
never SQL f-strings;
url_has_allowed_host_and_scheme(); CORS: django-
cors-headers.

CACHE

CACHES = {"default": {"BACKEND": "django.core.
cache.backends.redis.RedisCache",
"LOCATION": "redis://127.0.0.1:6379"},
cache.set("k", v, 300); cache.get_or_set("K", fn)
await cache.aget("K"); @cache_page(60 * 15)
Redis, PyMemcache, DB, file, locmem, dummy
@vary_on_cookie, { cache % }; per-site:
UpdateCacheMiddleware first,
FetchFromCacheMiddleware last.

EMAIL (MAILERS, 6.1)

MAILERS = {"default": {"BACKEND": "django.core.
mail.backends.smtp.EmailBackend",
"OPTIONS": {"host": "smtp.example.net"}}}
send_mail(subj, body, frm, [to], using="default")
m = EmailMultiAlternatives(subj, txt, frm, [to])
m.attach_alternative(html, "text/html"); m.send()
MAILERS replaces EMAIL_x. Console/file/locmem/dummy
backends; django-anymail; tests: mail.outbox.

BACKGROUND TASKS

from django.tasks import task # 6.0
@task(queue_name="emails")
def notify(review_id): ...
result = notify.enqueue(review_id=r.pk)
TASKS = {"default": {"BACKEND": "django.tasks.
backends.ImmediateBackend"}}
Celery 5: @shared_task; t.delay(pk)
Built-in Immediate/Dummy backends are for dev/tests; real
workers via a third-party backend (django-tasks) or Celery
(retries, beat, Flower, delay_on_commit). Pass IDs;
idempotent tasks.

SIGNALS, LOGGING, PAGINATION

@receiver(post_save, sender=Book)
def on_save(sender, instance, created, **kw): ...
LOGGING = {"version": 1, "handlers": {...},
"loggers": {"django.request": {...}}}
logger = logging.getLogger(__name__)
p = Paginator(qs, 25)
page = 0; get_page(request.GET.get("page"))
Signals: pre_save post_save pre_delete
m2m_changed user_logged_in; connect in ready(); last
router.loguser.exception().
AdminEmailHandler | AdminSends. AsyncPaginator (6.0);
paginate ordered QuerySets.

I18N, L10N & TIME ZONES

from django.utils.translation import gettext_lazy
gettext("Hi"), "Hi books", n)
{% translate "Hi" %} | blocktranslate %}..
./manage.py makemessages -l es; compilemessages
USE_TZ = True # default 5.0; store UTC
timezone.now(); zoneinfo.ZoneInfo("Europe/Madrid")
LocaleMiddleware, i18n_patterns, LANGUAGES,
set_language; timezone.activate(), { local time
%}. Never concatenate translated fragments.

CONTRIB APPS & CHECKS

contenttypes - sites, sitemaps, syndication (Feed)
flatpages redirects humanize - postgres:
ArrayField, SearchVector SearchQuerySet
TigrisSimilarity, GiniIndex - GeoDjango
GIS (PostGIS, PointField). Checks: @register() returning
Error/Warning; SILENCED_SYSTEM_CHECKS.

ASYNCH & CHANNELS

async def view(request):
book = await Book.objects.aget(pk=1)
asgi.py (Channels 4)
application = ProtocolTypeRouter({
"http": get_asgi_application(),
"websocket": AllowedHostsOriginValidator(
AuthMiddlewareStack(URLRouter([path("/ws/<w>/",
RoomConsumer.as_asgi())]))))
class RoomConsumer(AsyncWebsocketConsumer):
async def connect(self):
await self.channel_layer.group_add(
"room", self.channel_name)
await self.accept()
Async needs ASGI; sync middleware costs thread hops;
sync, to_async/async, to_sync,
database_sync, to_async.CHANNEL_LAYERS with
channels_redis; "daphne" first in INSTALLED_APPS.

WEB APIS: DRF 3.18 & NINJA 1.7

class BookSer(serializers.ModelSerializer):
class Meta:
model = Book; fields = ["id", "title"]
class BookViewSet(viewsets.ModelViewSet):
serializer_class = BookSer
permission_classes = [IsAuthenticatedOrReadOnly]
def get_queryset(self):
return Book.objects.filter(public=True)
router = DefaultRouter()
router.register("books", BookViewSet)
api = NinjaAPI()
class BookOut(Schema): # Django Ninja
class BookOut(schema): int: int; title: str
@api.get("/books", response=List(BookOut))
def books(request): return Book.objects.all()
path("api/", api.urls)
DRF: auth, throttling, pagination, django-filter, @action,
drf-spectacular, APITestCases. Ninja: Pydantic, async.
GraphQL: Strawberry-Django.

TESTING

class BookTests(TestCase): # rollback per test
@classmethod
def setUpTestData(cls):
cls.b = Book.objects.create(title="Django")
def test_list(self):
r = self.client.get(reverse("catalog:list"))
self.assertContains(r, "Django")
with self.captureOnCommitCallbacks(
execute=True): ...
SimpleTestCase (no DB), TestCase,
TransactionTestCase, StaticLiveServerTestCase
(Playwright/Selenium); Client AsyncClient
RequestFactory force_login() @override_settings
mail.outbox.assertRedirects assertTemplateUsed
assertQuerySetEqual pytest-django, factory_boy,
model-bakery.

DEPLOYMENT

gunicorn config.wsgi -w 4 # WSGI
gunicorn config.asgi:application \
-k uvicorn.worker.UvicornWorker # ASGI
uvicorn | daphne | granian config.asgi:application
./manage.py check --deploy; migrate; collectstatic
NGINX (TLS, static/media) or WhiteNoise; DEBUG=False,
env secrets, pool health checks; one process per
container; upgrade one release at a time (python -Wd,
django-upgrade).

Django is a registered trademark of the Django Software Foundation. This unofficial summary covers the official documentation (docs.djangoproject.com/en/6.1). Tested against Django 6.1.1 on 2026-10-03. irurueta.com